

IplaceX

enovus

Alexis Rosales González

PROGRAMACION HIBRIDA

Bimestre 5

Aplicación de avisos para comunidad de “Peor es nada”

Introducción

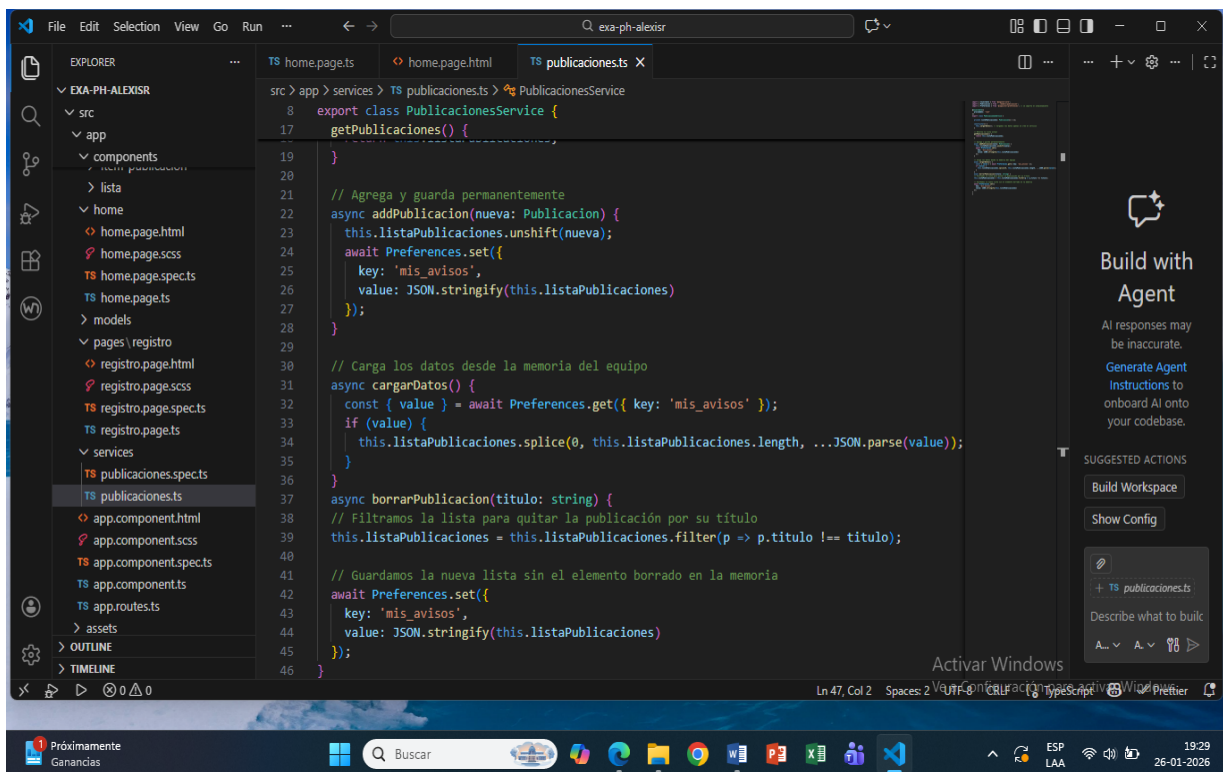
El presente proyecto consiste en el desarrollo de una aplicación móvil híbrida diseñada para la comunidad de “Peor es nada”, utilizando el framework Ionic con Angular. El principal objetivo es crear una plataforma que permita a los habitantes de la comunidad registrar avisos de utilidad pública, integrando capacidades de hardware como la cámara fotográfica y garantizando también la persistencia de los datos en el dispositivo. A continuación, presento los pilares técnicos y funcionales que le dan vida a este proyecto, explicando la lógica de cada implementación clave.

1. Gestión de Persistencia con Capacitor Preferences.

Para asegurar que los avisos no se pierdan al cerrar la aplicación, implementé un sistema de almacenamiento local. Utilice el plugin Preferences de Capacitor para serializar mi lista de objetos en formato JSON, permitiendo que la información de cada habitante de “Peor es Nada” se mantenga grabada de forma permanente en el dispositivo.

```
async addPublicacion(nueva: Publicacion) { this.listaPublicaciones.unshift(nueva);  
  
  await Preferences.set({ key: 'mis_avisos', value:  
    JSON.stringify(this.listaPublicaciones) });}
```

****// En esta sección se utiliza el método set de preferences para transformar el arreglo de publicaciones en una cadena de texto técnica que el almacenamiento del celular puede guardar de forma segura.****



“Figura 1: Implementación del servicio de almacenamiento persistente utilizando el plugin Preferences para la gestión de datos locales.”

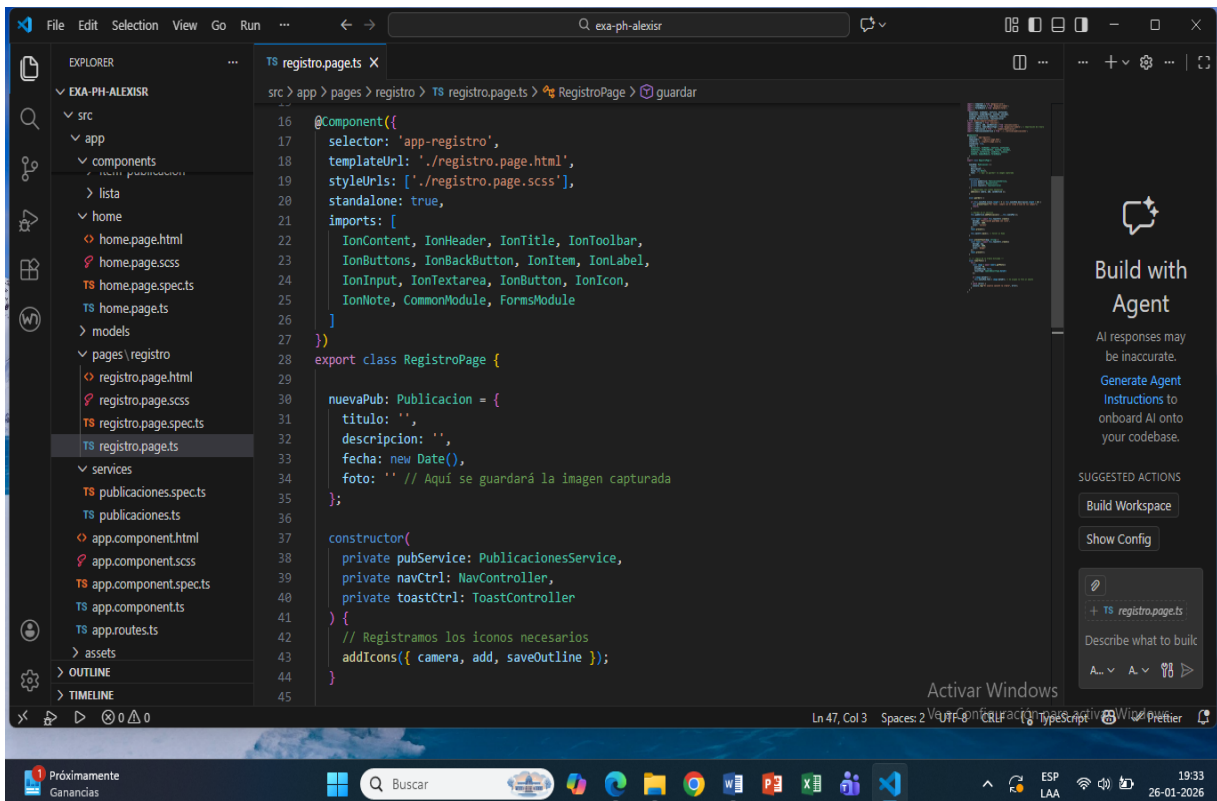
2. Inyección de Dependencias y Comunicación entre Componentes.

En este paso para que mi aplicación sea escalable y organizada, utilice la inyección de dependencias en los constructores de mis páginas. Esto me permitió conectar el servicio de publicaciones con la interfaz de usuario, facilitando que el registro así también como el home compartan la misma fuente de datos en tiempo real.

```
constructor( private pubService: PublicacionesService, private navCtrl:  
NavController, private toastCtrl: ToastController) {
```

```
  addIcons({ camera, add, saveOutline });
```

```
} /** En el constructor defino las herramientas que necesita mi página para  
funcionar, inyectando el servicio de publicaciones para que la lógica de negocio  
este separado de la vista./**
```

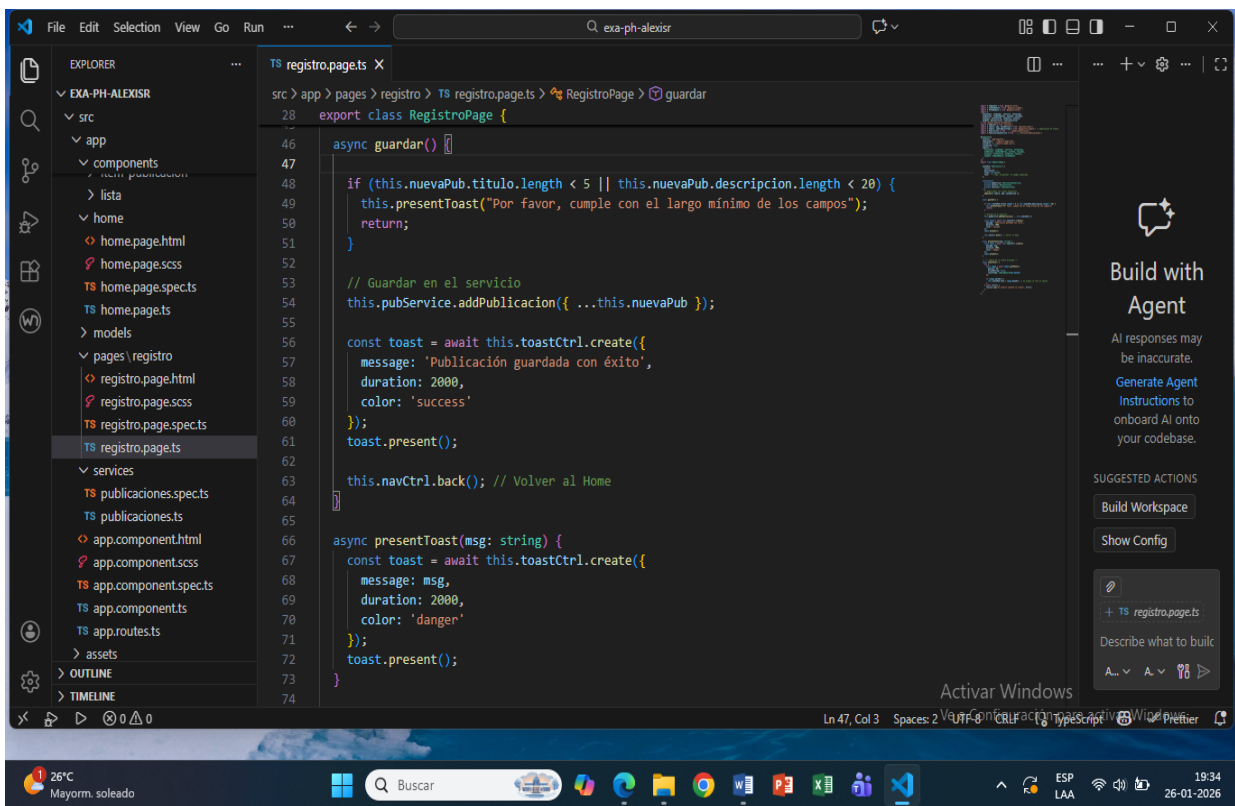


“Figura 2: Configuración del constructor y registro de iconos mediante la inyección de servicios esenciales para el funcionamiento del módulo.”

3. Lógica de Validaciones de Seguridad.

Aquí para mantener la integridad de la información, programé validaciones lógicas antes de permitir el guardado. Me asegure de que el sistema verifique que el título tenga al menos 5 caracteres y la descripción un mínimo de 20, protegiendo la base de datos de ingresos incompletos.

```
if (this.nuevaPub.titulo.length < 5 || this.nuevaPub.descripcion.length < 20) {  
  this.presentToast("Por favor, cumple con el largo mínimo de los campos");  
  
  return;  
}  
/**//Con este condicional garantizo que el usuario ingrese información suficiente  
antes de permitir que el servicio almacene el nuevo aviso en la base de datos  
local.//**
```

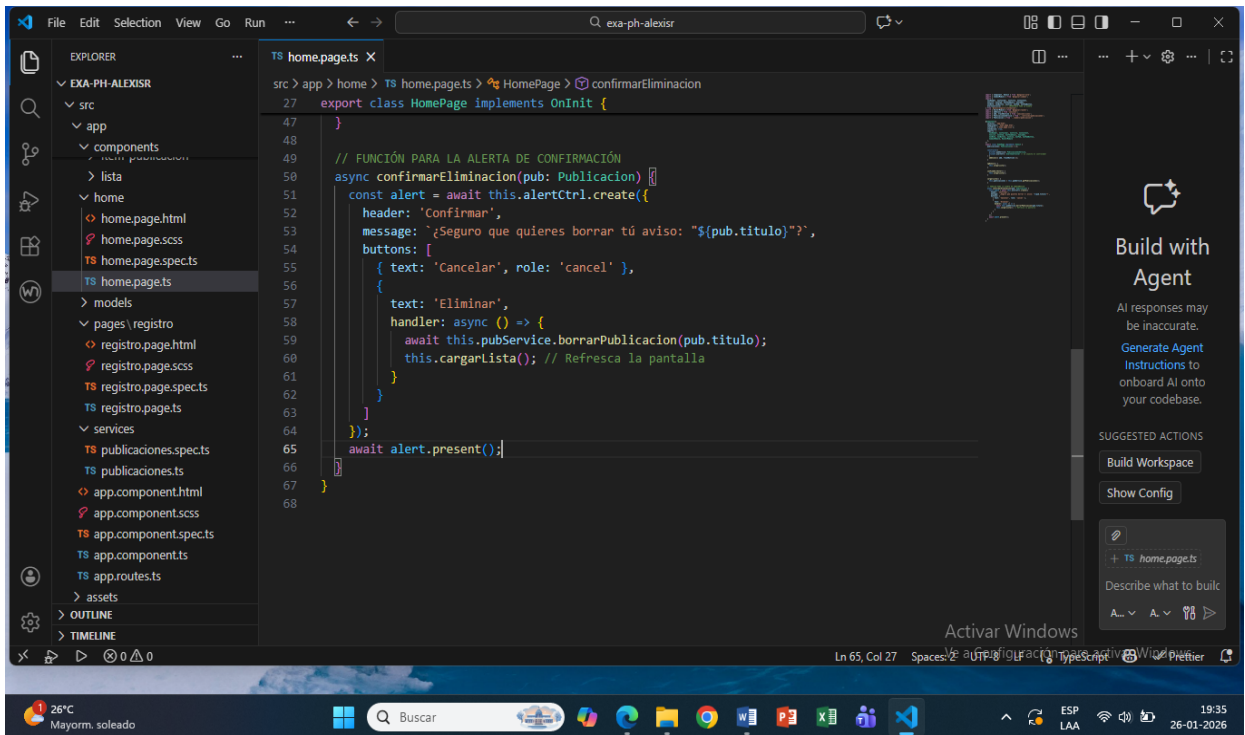


“Figura 3: Segmento del código que controla el flujo de guardado condicionado al cumplimiento de las extensiones mínimas del texto.”

4. Control de Diálogos Asincrónicos (AlertController).

Implemente una capa de seguridad para evitar eliminaciones accidentales mediante el uso de promesas. Utilice el AlerController para pausar la ejecución del código hasta que el usuario tome una decisión, garantizando un flujo de trabajo seguro en la gestión de avisos.

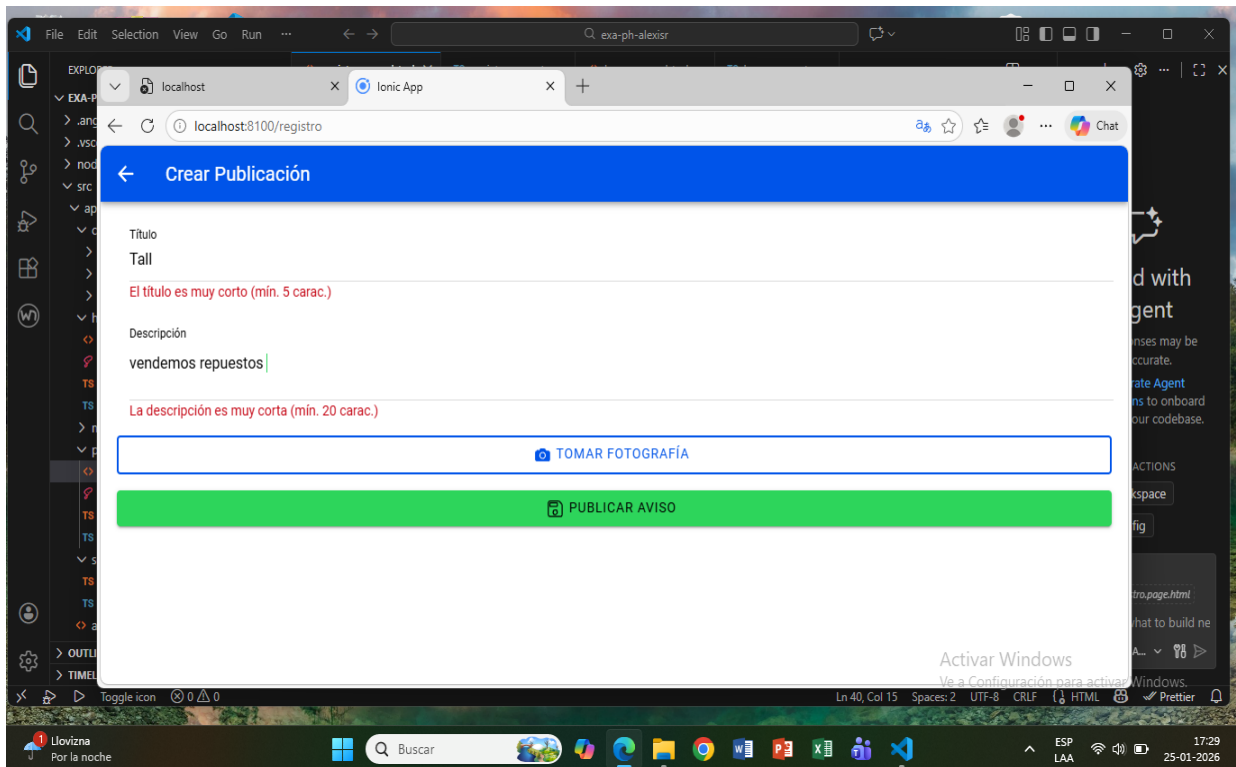
```
const alert = await this.alertCtrl.create({  
  
  header: 'Confirmar',  
  
  message: ¿Seguro que quieres borrar tú aviso: "${pub.titulo}"?,  
  
  buttons: [ ... ] }); await alert.present(); **// Se programa una alerta asincrónica que intercepta la intencion de borrado.**
```



“Figura 4: implementación técnica del componente de alerta para la validación de acciones críticas por parte del usuario”

5. Demostración de Restricciones y Mensajes de Error (Ejecución).

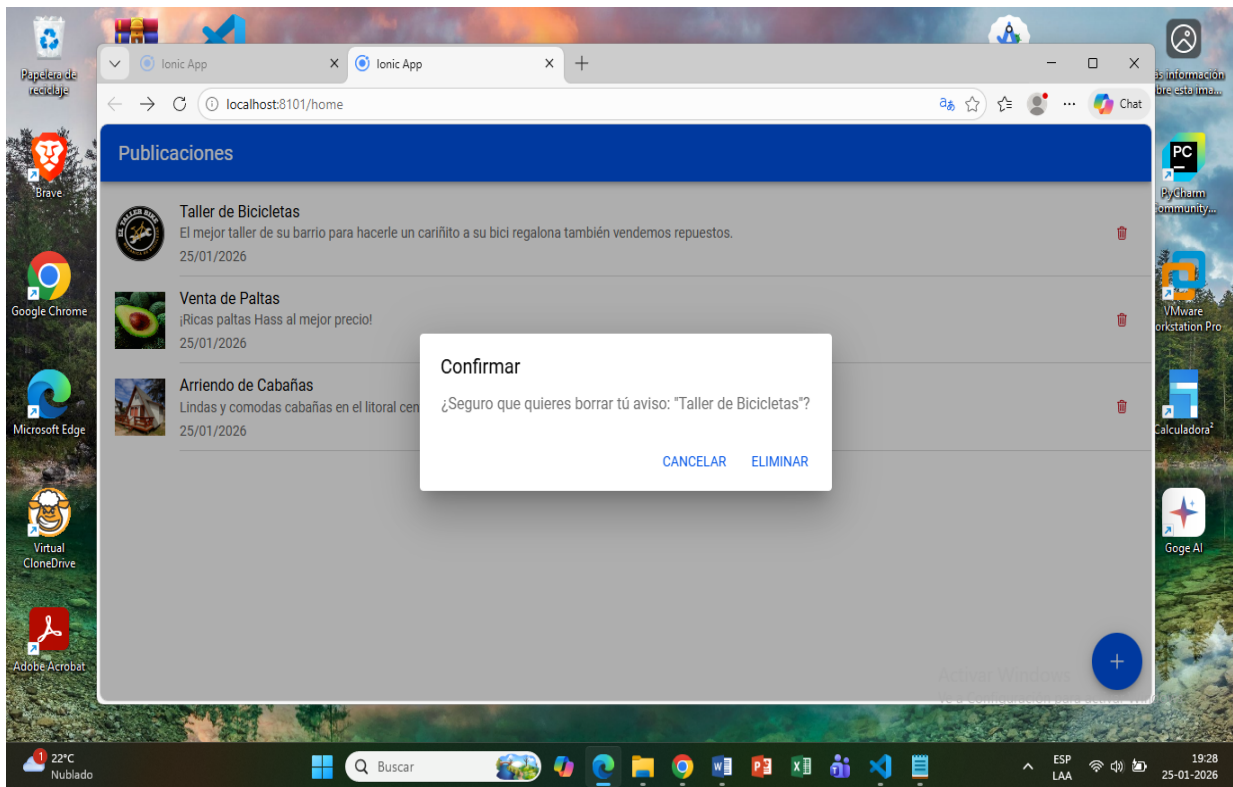
Prueba de ejecución, en donde se demuestra como la aplicación reacciona ante un uso incorrecto del formulario. Al intentar guardar un aviso que no cumple con el largo mínimo de 5 o 20 caracteres, el sistema bloquea el proceso y despliega un mensaje de color rojo, guiando al usuario para corregir la información.



“Figura 5: Validación en tiempo de ejecución donde el sistema impide el registro de datos insuficientes mediante un mensaje de advertencia.”

6. Visualización de lista y confirmación final (Ejecución).

Para finalizar, se presenta la interfaz final con los registros almacenados exitosamente, incluyendo sus imágenes y fechas. También se observa la activación de la alerta de eliminación personalizada con el nombre del aviso, confirmando que la integración entre el servicio y la vista.



“Figura 6: interfaz de usuario operativa mostrando el listado de los avisos y la funcionalidad de borrado con validación de identidad del registro.”